Makemore

Part 3 - Activations and Gradients, BatchNorm

Why initialization matters, and how Batch Normalization saves us from it

---

A comprehensive, cell-by-cell study handbook for the third video in Andrej Karpathy's *makemore* series — “Building makemore Part 3: Activations & Gradients, BatchNorm.” This expanded edition explains the **what**, the **why**, and the **how** behind every concept, code cell, and diagnostic plot in the notebook.

Compiled by: ***Shreyas Pradeepkumar Khandale*** · *GitHub:* ***sherurox***

Companion to: `makemore_part3_bn.ipynb`

Edition: Expanded - May 2026

Where this handbook sits in the series

Part 2 built a multi-layer perceptron that took three previous characters as context, embedded each into a 10-dimensional learned vector, passed the concatenated embeddings through a `tanh` hidden layer of 200 units, and finally projected to 27 logits. Trained with minibatch SGD and a step-down learning rate, the network reached a dev loss of about **2.17**, beating the bigram baseline of 2.45. The architecture was correct, the math was right, the loss was improving — everything looked fine on the surface.

Part 3 takes that exact same network and zooms in on the internals. It does *not* change the architecture, the dataset, or the objective. What it changes is the way the parameters are *initialized* at the start of training, and it introduces a new layer — **Batch Normalization** — that controls the distribution of activations *throughout* training. Both changes are motivated by a single observation: the network we built in Part 2 is actually quite badly configured at the start, and we can prove this just by looking at the loss in the very first iteration.

What is actually new compared to Part 2

- **Initialization theory.** We learn how to predict the expected initial loss from first principles, why “confidently wrong” outputs make the first epoch wasteful, and how to fix it with a simple scale on the output layer.
- **The saturated tanh problem.** We visualize how the hidden pre-activations are spread far too wide, causing most of the `tanh` outputs to collapse to ±1 where the gradient is essentially zero.

- **Dead neurons.** We introduce a phenomenon shared by `tanh`, sigmoid, and ReLU: neurons that never activate and therefore never learn. The mechanism, the diagnosis, and the fix.
- **Kaiming initialization.** The principled formula from He et al. 2015 that replaces our hand-tuned scaling factors. We work out the gain of 5/3 for `tanh`.
- **Batch Normalization.** The 2015 Ioffe & Szegedy paper that made deep networks routinely trainable. The forward pass, the learnable gain and shift, the running statistics for inference, and the caveats.
- **PyTorchifying.** Wrapping the operations into `Linear`, `BatchNorm1d`, and `Tanh` classes that mirror the PyTorch `nn.Module` API. The 6-layer deep MLP we build with them.
- **The diagnostic toolkit.** Activation histograms, gradient histograms, the weight-gradient distribution, and the all-important update-to-data ratio plot.

Preface: How to use this handbook

This handbook is the third in a coordinated series covering Andrej Karpathy's *makemore* lectures. It is organized as a guided walk through the `makemore_part3_bn.ipynb` notebook, with each chapter mapping to a coherent group of cells in the notebook. The format is identical to the Part 1 and Part 2 expanded editions so the three documents can be read side-by-side as one continuous reference.

Reading mode 1: Linear study

Read the handbook end-to-end while keeping the notebook open in a separate window. Each chapter introduces the concept, presents the relevant code cell exactly as it appears in the notebook, and then explains what the code is doing, why it is structured that way, and how it fits into the larger story. This is the recommended path the first time you study the lecture, because Part 3 is much more diagnostic and intuition-driven than the earlier parts — the code is shorter, but understanding *why* we run each diagnostic is what makes the lesson stick.

Reading mode 2: Reference lookup

Once you have completed a first pass, the handbook also functions as a reference manual. The Table of Contents lists every concept with its chapter number, and the Glossary at the end provides one-line definitions for every technical term used in the lecture. The diagnostic cards in Chapter 28 are intentionally compact — consult them whenever you suspect your network is misbehaving in a new project, even years from now.

Conventions used in this handbook

The handbook uses a small set of visual conventions consistently throughout the document. Recognizing them will let you scan and read faster.

- **Code blocks** appear in a fixed-width font on a warm beige background. Code is reproduced verbatim from the notebook so you can match each block to the cell that produced it.
- **Cell labels** precede every code block in small blue type (for example, *“Cell — The forward pass with BatchNorm.”*).
- **Callout boxes** (yellow background, rust-coloured title) highlight Karpathy's spoken framing, important warnings, or moments of insight.
- **Info boxes** (light blue background) contain conceptual definitions and theoretical background. They are useful for first-time learners and can be skimmed by readers already familiar with the material.
- **Part dividers** separate the five major sections of the handbook with a full-width banner.
- **Diagnostic cards** (Chapter 28) present each common training pathology in its own clearly bounded card with three labeled rows: symptom, diagnosis, and action.

Prerequisites

This handbook assumes you have completed the Part 1 and Part 2 handbooks (or are comfortable with the bigram model, the MLP architecture, the embedding layer, `F.cross_entropy`, minibatch SGD, the train/dev/test split, and how to read the train/dev loss gap). Mathematical prerequisites are modest: you should be comfortable with the chain rule, variances and standard deviations of random variables, and the idea that scaling a tensor by a constant scales its standard deviation by the same constant. We do not derive the Kaiming formula rigorously, but we explain enough of the intuition that you can recognize when to apply it.

How to extract maximum value from this material

Part 3 is the most diagnostic of the makemore lectures. The single most valuable habit you can build from it is the discipline of inspecting your network — checking initial loss against the theoretical baseline, plotting activation and gradient histograms, watching the update-to-data ratio — *before* assuming the model is working. The plots and diagnostics in this handbook are not optional decoration. They are the toolkit you will use on every neural network you ever debug. Type out each diagnostic block yourself, look at the resulting plot, and verbalize what a healthy plot would look like before you trust the numbers.

Contents

PART I

The starting point

Where the lecture begins: the Part 2 MLP, slightly cleaned up, ready to be diagnosed and improved.

1. Where Part 3 picks up in the series

Chapter overview

Karpathy opens the lecture with a deliberate framing: although the natural next architectural step would be to introduce recurrent neural networks, he is going to stay at the multi-layer perceptron level for one more lecture. The reason is that what we are about to learn — how to think about the statistics of activations and gradients during training — is the prerequisite for understanding why deeper networks are so hard to train, and why the modern innovations (better initialization, batch normalization, residual connections, modern optimizers) were invented. This chapter explains the motivation and sets expectations for what the lecture will and will not cover.

We just finished Part 2: we have a working character-level MLP that takes three previous characters as context and predicts the next one, with a dev loss of around 2.17. The architecture is correct and the training loop works. So why are we still on the MLP, instead of moving forward to RNNs?

> **Why stay on the MLP for another lecture**
> Recurrent neural networks are universal approximators in principle, but they are notoriously hard to train with first-order gradient methods. The reason has nothing to do with their expressive power and everything to do with the statistics of activations and gradients as they propagate through many layers. The same intuitions apply to deep MLPs, deep CNNs, and Transformers. Karpathy uses the Part 2 MLP as a controlled environment in which to develop these intuitions, because the architecture is simple enough that we can inspect every layer and every gradient by hand. Once these intuitions are solid, all subsequent architectures look familiar.

What this lecture is really about

Part 3 is not primarily about improving the loss number — in fact, the final dev loss is only marginally better than Part 2's. The real subject is the *internals* of a training neural network: the distribution of pre-activations and activations, how the gradients propagate backward through the network, what a healthy training run looks like under inspection, and how to tell the difference between a model that is training well and one that is silently broken. These are the skills that distinguish someone who can copy-paste a training script from someone who can actually debug a neural network.

Three goals of the lecture

- **Introduce Batch Normalization**, one of the first “modern” innovations that made very deep networks routinely trainable. Published in 2015 by Ioffe and Szegedy, it is still in every modern toolbox.
- **PyTorch-ify the code.** Wrap the operations into reusable classes (`Linear`, `BatchNorm1d`, `Tanh`) that mirror PyTorch's `nn.Module` API. After this lecture, the code looks like real PyTorch.

- **Introduce the diagnostic toolkit.** Activation histograms, gradient histograms, weight-gradient distribution, and the update-to-data ratio. These are the lenses you will use on every neural network you train from this point onward.

2. Setup recap: imports, data, train/dev/test

Chapter overview

The first several cells of the notebook are essentially identical to Part 2: import PyTorch and matplotlib, read `names.txt`, build the `stoi`/`itos` mappings, and construct the three train/dev/test splits using the same windowing function and the same random seed. This chapter walks through these cells briefly. If you have done Part 2, you can skim it — we will spend our energy on the new material.

Cell - Imports

```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt
%matplotlib inline
```

Cell - Read the dataset and build the vocabulary

```
words = open('names.txt', 'r').read().splitlines()
words[:8]
len(words)              # 32033

chars = sorted(list(set(''.join(words))))
stoi = {s: i+1 for i, s in enumerate(chars)}
stoi['.'] = 0
itos = {i: s for s, i in stoi.items()}
vocab_size = len(itos)
print(itos)
print(vocab_size)       # 27
```

One small change worth noting: we now store `vocab_size` as a named variable instead of hard-coding 27 throughout the notebook. Karpathy is doing this throughout Part 3 — every magic number from Part 2 is being pulled out into a named hyperparameter at the top of the notebook, so the code can be re-run with different choices without hunting through every line.

Cell - Build the train / dev / test splits

```
block_size = 3  # context length

def build_dataset(words):
    X, Y = [], []
    for w in words:
        context = [0] * block_size
        for ch in w + '.':
            ix = stoi[ch]
            X.append(context)
            Y.append(ix)
            context = context[1:] + [ix]
    X = torch.tensor(X)
    Y = torch.tensor(Y)
```

```
    print(X.shape, Y.shape)
    return X, Y

import random
random.seed(42)
random.shuffle(words)
n1 = int(0.8 * len(words))
n2 = int(0.9 * len(words))

Xtr,  Ytr  = build_dataset(words[:n1])     # 80%
Xdev, Ydev = build_dataset(words[n1:n2])   # 10%
Xte,  Yte  = build_dataset(words[n2:])     # 10%
```

Identical to Part 2. Three context characters, sliding window construction, 80/10/10 split with `random.seed(42)` for reproducibility. The training set is now about 182,000 examples, dev and test are about 22,000 each. If anything in this block is unfamiliar, review Part 2 Chapter 23 before continuing.

3. The refactored baseline MLP

Chapter overview

The next cell reconstructs the Part 2 model with a couple of cosmetic improvements: hyperparameters are pulled out into named variables (`n_embd`, `n_hidden`), the parameter list construction is explicit, and the parameter count is printed for inspection. Functionally nothing has changed yet. This is our baseline — the model we are about to diagnose and improve.

Cell - Initialize the baseline MLP

```
n_embd = 10      # dimensionality of the character embedding vectors
n_hidden = 200   # number of neurons in the hidden layer

g = torch.Generator().manual_seed(2147483647)
C  = torch.randn((vocab_size, n_embd),            generator=g)
W1 = torch.randn((n_embd * block_size, n_hidden), generator=g)
b1 = torch.randn(n_hidden,                        generator=g)
W2 = torch.randn((n_hidden, vocab_size),          generator=g)
b2 = torch.randn(vocab_size,                      generator=g)

parameters = [C, W1, b1, W2, b2]
print(sum(p.nelement() for p in parameters))   # ~11,897
for p in parameters:
    p.requires_grad = True
```

Same five parameter tensors as Part 2. About 12,000 parameters in total. Each was drawn from a standard normal distribution with the canonical seed `2147483647`, just as before. Note that at this point we have *no* scaling factors on the weights yet — those are what Part 3 is about to introduce.

How the hyperparameter extraction will pay off later

Pulling the dimensions out into `n_embd` and `n_hidden` looks like trivial refactoring, but it will be essential once we start the PyTorch-ifying section in Chapter 21. There we build a six-layer network whose hidden

width is determined by a single number; if the original code had 200 hardcoded in five places, we would have to edit five lines every time we wanted to experiment. Named hyperparameters concentrate the configuration in one place.

Cell - The same training loop as Part 2

```
max_steps = 200000
batch_size = 32
lossi = []

for i in range(max_steps):
    # minibatch construct
    ix = torch.randint(0, Xtr.shape[0], (batch_size,), generator=g)
    Xb, Yb = Xtr[ix], Ytr[ix]

    # forward pass
    emb = C[Xb]
    embcat = emb.view(emb.shape[0], -1)
    hpreact = embcat @ W1 + b1
    h = torch.tanh(hpreact)
    logits = h @ W2 + b2
    loss = F.cross_entropy(logits, Yb)

    # backward pass
    for p in parameters: p.grad = None
    loss.backward()

    # update
    lr = 0.1 if i < 100000 else 0.01
    for p in parameters:
        p.data += -lr * p.grad

    if i % 10000 == 0:
        print(f'{i:7d}/{max_steps:7d}: {loss.item():.4f}')
    lossi.append(loss.log10().item())
```

The loop is unchanged from Part 2. Note three details that will matter shortly: we explicitly compute `hpreact` (the "pre-activation", the value going into `tanh`) as its own variable, we log `log10(loss)` so the curve is readable, and we drop the learning rate by 10× after 100,000 steps.

> **What is hpreact and why isolate it?**
> `hpreact` is the value of `embcat @ W1 + b1` *before* the `tanh` non-linearity is applied. We give it its own name in Part 3 because it is the quantity we are about to scrutinize and modify. Most of the diagnostic work in this lecture is about the statistical properties of `hpreact` — its mean, its standard deviation, and the resulting shape of the `tanh` output. Naming it gives us something concrete to point at.

After running the loop, the validation loss is about 2.17 — same as we ended Part 2 with. Everything is working. There are no bugs. And yet, as we are about to see, the network is in a deeply unhealthy state during the first several thousand iterations. We just had not looked closely enough to notice.

PART II

Diagnosing initialization

Why the network we built in Part 2 is badly configured at the start, and how to fix it in two principled steps.

4. The first symptom: a hockey-stick loss curve

Chapter overview

Karpathy points out that if you look at the printed loss values from the training loop — or better, plot `lossi` on a graph — you see something suspicious. The very first iteration records a loss of around **27**, which then crashes down to around 1 or 2 within the first few hundred steps, and only after that does the slow, steady descent toward the final loss begin. The shape of this curve resembles a hockey stick — a steep early drop followed by a long shallow tail — and it is a clear sign that something is wrong at initialization.

Cell - Plot the loss

```
plt.plot(lossi)
```

The plot shows a very tall, very narrow spike at iteration 0 dropping rapidly into the noisy band that constitutes the normal training trajectory. The visual is striking: it looks like the first few hundred iterations are spent fixing a problem that should not have existed in the first place.

> **Why the hockey stick is a red flag**
> A neural network at the very start of training should output predictions that are essentially uniform random — it has not learned anything yet, so it should have no opinion about which character comes next. The loss corresponding to a uniform distribution is a known constant we can compute from first principles. When the observed initial loss is much higher than that constant, the network is not in a neutral state; it is in a state of confident wrong opinion, and the first epoch or so of training is wasted just undoing that misconfiguration. The hockey stick is the visual signature of this wasted training.

5. What initial loss should we expect?

Chapter overview

Before we diagnose the wrong initial loss, we need to know what the *right* initial loss is. This chapter computes it from first principles. The calculation is short and worth memorising: it applies to every classification model you will ever build.

At initialization, the network has no information about which character is more likely than any other. The only sensible behaviour is a uniform distribution over the 27 possible outputs — each character should be assigned probability 1/27.

Cell - The expected initial loss

```
# At initialization, each character should have probability 1/27
-torch.tensor(1/27.0).log()
# tensor(3.2958)
```

The expected initial loss is therefore –**log(1/27) = log(27)** ≈ **3.29**. Any reasonable initialization should start the loss near this value. Our network started at 27 instead. The discrepancy — an order of magnitude larger than expected — is what we are going to fix.

Why this calculation generalizes

The same logic applies to any classification problem with *K* output classes. The expected initial loss is always **log(K)**. For binary classification (K=2), the expected initial loss is log(2) ≈ 0.69. For ImageNet (K=1000), it is log(1000) ≈ 6.91. For a Transformer with a 50,000-token vocabulary, it is log(50000) ≈ 10.82. Whenever you start training a classification model, calculate log(K) and compare it to the first loss value you see. If they disagree by more than a small factor, your initialization is broken, and fixing it before doing anything else will save you hours of wasted training time.

6. The "confidently wrong" phenomenon

Chapter overview

Knowing that the expected initial loss is 3.29 and the observed initial loss is 27, we now have to explain the gap. This chapter walks through the mechanism: the random initialization assigns large magnitudes to the logits, the softmax converts those large magnitudes into very sharp probability distributions, and the network ends up assigning high probability to incorrect characters and tiny probability to the correct ones. Karpathy calls this state **“confidently wrong.”**

The output of the network's final linear layer is `logits = h @ W2 + b2`. At initialization, `W2` and `b2` are drawn from `randn`, so their entries have standard deviation 1. The resulting logits have a range of roughly ±5 to ±10 across the 27 classes for any given example.

What does softmax do to logits with that range? Consider a tiny example with four logits `[0, 5, −3, 2]`. After exponentiation we get `[1, 148, 0.05, 7.4]`. After normalization, almost all the probability mass (roughly 94%) lands on the second class, with the others getting tiny shares. The network is extremely confident that the answer is class 2 — but at initialization there is no reason for that confidence. If the true label happens to be one of the other classes, the loss `−log(prob)` can be enormous, because `prob` is tiny.

> **Why confidently wrong is much worse than uniformly wrong**
> If the network were uniformly uncertain (each probability is 1/27), every example would contribute loss log(27) ≈ 3.29, regardless of which class was correct. But when the network is sharp and wrong, examples where the true class happened to get a tiny probability contribute losses of 10, 20, or more — while examples where the true class happened to get a large probability contribute almost zero loss. The average of these is much higher than log(27), because the rare bad examples dominate.

The smaller, four-dimensional example Karpathy walks through in the video makes this concrete. With logits all near zero, softmax produces a uniform distribution and the loss is exactly `log(4) = 1.386` regardless of the label. With logits like `[0, 5, 0, 0]`, the second class is heavily favoured. If the true label is also 2, the loss is tiny. If the true label is 0, 1, or 3, the loss is around 5. Averaged over random labels, this is much worse than the uniform baseline.

Karpathy's framing

“The network is confidently wrong — some characters are very confident and some are very not confident. That's what makes it record a very high loss.” The cure is to make the network *diffidently uniform* at initialization, so that the first iteration loss is close to log(K) and all subsequent training is spent learning real patterns rather than dialing down random overconfidence.

7. Fix 1: squashing the output layer

Chapter overview

The fix for confidently-wrong initialization is mechanical and direct: make the logits small at the start. We achieve this by scaling down the output layer's weights and zeroing out its biases. After this fix, the network outputs a near-uniform distribution at initialization, the first loss is very close to log(27), and the hockey stick disappears from the loss curve.

The change is two characters and a multiplication:

Cell - Initialize the output layer with smaller weights and zero bias

```
C  = torch.randn((vocab_size, n_embd),            generator=g)
W1 = torch.randn((n_embd * block_size, n_hidden), generator=g)
b1 = torch.randn(n_hidden,                        generator=g)
W2 = torch.randn((n_hidden, vocab_size),          generator=g) * 0.01
b2 = torch.randn(vocab_size,                      generator=g) * 0
```

Why this works

`W2` is multiplied by 0.01 (and `b2` by 0, which sets it to a zero tensor). At initialization, every logit is now a sum of products of small numbers from h and from `W2 * 0.01`. The logits cluster tightly around zero, with standard deviation around 0.01 times what they were before. The softmax of near-equal logits is nearly uniform, every class gets about 1/27, and the loss is about 3.29 — exactly the theoretical baseline.

Why we do not set the weights to exactly zero

It is tempting to ask: if we want uniform probabilities at init, why not just set `W2 = 0`? The answer is that exact zero weights would make all the logits exactly equal across examples too, which removes any signal from h for the first few gradient steps. By scaling rather than zeroing, we preserve a small amount of entropy — tiny but nonzero differences between examples — which gives the gradient something to grab onto right away. The bias `b2` can be safely zeroed because it adds the same constant to every class, which cancels out in softmax.

The first improvement

After this fix, training proceeds without the wasteful early hockey stick. The first loss is around 3.29 as predicted, the curve descends smoothly, and we end up with a validation loss of about **2.13**, down from 2.17. That is a meaningful improvement from a two-line change at initialization.

8. The second symptom: visualizing the hidden activations

Chapter overview

The output-layer fix solves one problem, but Karpathy is not done. He now turns the spotlight on the *hidden* layer — specifically, on the distribution of `h`, the post-tanh activations — and finds a second pathology that is independent of the first. Most of the hidden activations are saturated at ±1, which means most of the gradients are effectively zero. This chapter introduces the diagnostic that reveals the problem.

The diagnostic is a histogram of `h.view(-1).tolist()`: collect every value of every hidden unit across the batch, flatten into one list, and plot the distribution.

Cell - Histogram of h (post-tanh)

```
plt.hist(h.view(-1).tolist(), 50);
```

The result is a U-shaped distribution: two giant spikes near `+1` and `–1`, with very few values in between. In other words, almost every hidden unit on every example is either fully on (1) or fully off (–1). The `tanh` is **saturated**.

The complementary visualization: a 2-D image of where saturation occurs

Cell - Visualize the saturation map

```
plt.figure(figsize=(20, 10))
plt.imshow(h.abs() > 0.99, cmap='gray', interpolation='nearest')
```

This produces a (batch_size × n_hidden) image where every pixel is white if `|h| > 0.99` (saturated) and black otherwise. The image is mostly white — the network is heavily saturated for most examples on most neurons. Worse, the presence of any neuron whose *entire column* is white would mean that neuron is saturated for every example in the batch, in which case it can never learn. This is the spectre of the dead neuron, which the next chapter explains.

Why tanh saturation matters

The derivative of `tanh(x)` is `1 − tanh(x)²`. When `tanh(x)` is close to +1 or −1, that derivative is close to zero. During backpropagation, the gradient flowing back through a saturated tanh gets multiplied by approximately zero, which means no gradient signal reaches the parameters of any preceding layer. The neurons in those preceding layers do not update, and the network learns very slowly.

9. Why saturated tanh kills gradients

Chapter overview

We saw in Part 2 that the chain rule of backpropagation multiplies gradients layer-by-layer as the signal flows from loss back toward the parameters. This chapter zooms in on what happens at a tanh node when its output is saturated, and explains why this is the central pathology of poorly initialized deep networks.

Consider a single `tanh` neuron. Its forward computation is `h = tanh(hpreact)`. Its backward pass, by the chain rule, is `dhpreact = dh * (1 − h²)`. Now imagine the network has settled on an initialization where `h` is consistently 0.99 or −0.99. Then `1 − h² = 1 − 0.9801 = 0.0199`. Every gradient passing through that neuron is multiplied by 0.02 — a 50-fold attenuation. Stack two such layers and you have multiplied by 0.0004. Stack five and the gradient has essentially vanished.

The visual heuristic Karpathy uses

In the saturation-map image, every white pixel corresponds to a sample-neuron pair where the gradient is essentially zero. If a particular column is mostly white, that neuron rarely sees a usable gradient, so it learns slowly. If a column is *entirely* white — saturated for every example in the batch — that neuron will never see a non-zero gradient and will never learn. We have a dead neuron on our hands.

What Karpathy looks for

“We are looking for columns that are completely white. If any neuron is saturated for every example, no gradient will ever flow into it, and that neuron is dead — it will never learn. I do not see a single fully white column here, so we are okay — but we are not in a good place. Most neurons are saturated for most examples, and that is wasting most of the training signal.”

10. The dead neuron problem (tanh, sigmoid, ReLU)

Chapter overview

Saturation is not unique to `tanh`. Every non-linearity that has a flat region in its graph can produce dead neurons. This chapter generalizes the saturation concept to sigmoid and to ReLU — the two other most common activation functions — and explains why ReLU's dead-neuron failure mode is particularly insidious.

Tanh and sigmoid: dead from saturation

Both `tanh` and sigmoid are squashing non-linearities. Tanh saturates at ±1 with derivative approaching zero; sigmoid saturates at 0 and 1 with the same problem. A neuron with one of these activations is dead when, for every example in the dataset, its pre-activation falls in the flat-tail region. Once dead, no gradient reaches its weights and it never recovers.

ReLU: dead from the flat lower region

ReLU is `max(0, x)`. Its graph has a flat region for `x < 0` where the derivative is exactly zero, and a passing-through region for `x > 0` where the derivative is exactly one. A ReLU neuron is dead when its pre-activation is negative for every example in the dataset — the output is always zero, and the gradient flowing through it is always exactly zero. Unlike tanh's *nearly-zero* gradient, ReLU's dead-region gradient is *exactly* zero. There is no leakage. Once dead, a ReLU neuron is gone for good.

> **Permanent brain damage**
> Karpathy describes a dead ReLU neuron as “permanent brain damage in the mind of a network.” Even on a well-initialized network, a single training step with a too-large learning rate can knock a ReLU neuron into the dead region permanently. After enough training, it is common to find that 10-20% of the ReLU neurons in a network are dead. Variants like Leaky ReLU and ELU were introduced specifically to mitigate this: they have a non-zero (but small) slope in the negative region, so the neuron can still receive gradients and reactivate. PReLU learns the slope as a parameter.

How dead neurons happen

- **At initialization.** Bad weight scales push the pre-activations far from the active region of the non-linearity. Every example lands in the dead region; the neuron never activates and never learns.
- **During training, from a too-large learning rate.** A single overshooting gradient step pushes the weights into a regime where the pre-activation is permanently in the dead region for every input.
- **From data shift.** If the training data distribution changes mid-training (rare in supervised learning, common in reinforcement learning), neurons that used to activate may stop activating. They die silently.

11. Fix 2: squashing the hidden pre-activations

Chapter overview

We diagnosed the saturated tanh. The fix is the same idea as the output-layer fix from Chapter 7, applied one layer earlier: scale down `W1` and `b1` so that `hpreact` has a smaller standard deviation and falls into the active region of `tanh` rather than the saturated tails. Karpathy initially picks the scaling factor by eyeballing the resulting distribution; the next chapter replaces this with a principled formula.

Cell - Hand-tuned hidden-layer initialization

```
W1 = torch.randn((n_embd * block_size, n_hidden), generator=g) * 0.2
b1 = torch.randn(n_hidden,                        generator=g) * 0.01
```

`W1` is multiplied by 0.2 (chosen by trying 0.1 and finding it too small, then 0.2 and finding it about right), and `b1` is multiplied by 0.01 to leave a small amount of variation across neurons. The number 0.2 is not principled here — Karpathy explicitly says he is guessing-and-checking by looking at histograms. The principled version, Kaiming initialization, is what Chapter 13 introduces.

What the new histograms look like

After this change, the histogram of `hpreact` is roughly Gaussian with a standard deviation around 1, well inside the active region of `tanh`. The histogram of `h = tanh(hpreact)` is now spread across the entire (–1, 1) range, not piled up at the endpoints. The saturation map is mostly dark with only scattered white pixels — no fully white columns, and only a small fraction of pixels saturated at all. This is what a healthy activation distribution looks like.

> **The second improvement**
> After this fix, the validation loss drops from 2.13 to **2.10**. Combined with the output-layer fix from Chapter 7, we have improved from the Part 2 baseline of 2.17 down to 2.10 — entirely by adjusting initialization scales. The architecture is identical. We did not add a single new parameter. We just stopped wasting training on the wrong things.

Why hand-tuning will not scale

We just guessed our way to a working initialization, but the procedure was: look at a histogram, tweak a scaling factor, look again, tweak again. This works for our small two-layer network. It does not work for a 50-layer Transformer where every layer has its own scaling factor and the interactions compound non-linearly. We need a formula. That is what the next chapter delivers.

PART III

Principled initialization and Batch Normalization

From hand-guessed scaling factors to the Kaiming formula, and from there to the 2015 idea that made deep networks routinely trainable.

12. The preserve-the-std-across-layers principle

Chapter overview

The hand-tuning of Chapter 11 succeeded because we accidentally rediscovered a principle that has a clean mathematical statement: at initialization, every layer of the network should preserve the standard deviation of its input. If the input has standard deviation 1, the output should also have standard deviation 1. This chapter explains why this principle is so important and motivates the formula in the next chapter.

Consider a linear layer `y = x @ W`, where `x` has shape (batch, fan_in) and `W` has shape (fan_in, fan_out). Each output element `y[i, j]` is a sum of `fan_in` products `x[i, k] * W[k, j]`. If both `x` and `W` are independent Gaussians with mean 0 and standard deviation 1, then the standard deviation of the sum grows as `sqrt(fan_in)`.

> **Why standard deviation grows with fan_in**
> The variance of a sum of independent random variables is the sum of their variances. If we sum fan_in products each with variance 1, the total variance is fan_in. Taking the square root gives the standard deviation: `sqrt(fan_in)`. So a linear layer with fan_in = 30 amplifies the standard deviation by `sqrt(30) ≈ 5.5`. Stack two such layers and you have multiplied by $5.5^2 \approx 30$. After 10 layers, the magnitude has grown by a factor of about $10^{\cdot}$ — the activations are huge, the gradients vanish or explode, and the network cannot train.

The remedy: scale W to cancel the growth

If multiplying by W normally amplifies the standard deviation by `sqrt(fan_in)`, we can cancel this growth by dividing W by `sqrt(fan_in)`. The output of the layer then has the same standard deviation as the input. Stack as many such layers as you like and the magnitude does not blow up or shrink. This is the heart of modern initialization.

Why the tanh complicates the picture

The simple `1/sqrt(fan_in)` scaling assumes the activations are passing through a linear layer with no non-linearity. In practice, every linear layer is followed by a non-linearity such as `tanh`, sigmoid, or ReLU. Each of these non-linearities *contracts* the standard deviation by some characteristic factor: tanh contracts by about 0.6, ReLU by about 0.5 (because it zeros out half the values). To preserve the standard deviation across an entire linear + non-linearity block, we need to multiply by a **gain** factor that compensates for this contraction. This is what the Kaiming formula introduces in the next chapter.

13. Kaiming initialization explained

Chapter overview

Kaiming initialization, also known as He initialization after its inventor Kaiming He, was introduced in the 2015 paper *Delving Deep into Rectifiers*. The formula is simple: at initialization, draw the weights from a

distribution with standard deviation `gain / sqrt(fan_in)`, where `gain` depends on the non-linearity that follows. This chapter shows the formula in code and explains every term.

The formula in code form, using PyTorch's built-in:

Cell - PyTorch's kaiming-init helper

```
# Kaiming normal initialization with gain for tanh
std = (5/3) / (n_embd * block_size)**0.5   # gain / sqrt(fan_in)
W1 = torch.randn((n_embd * block_size, n_hidden), generator=g) * std
```

Or equivalently, the principled hand-coded form:

```
# In full:
# gain   = 5/3       (the multiplicative gain associated with tanh)
# fan_in = 30        (n_embd * block_size = 10 * 3)
# std    = 5/3 / sqrt(30) ≈ 0.304
# multiply randn by this std to scale appropriately
```

Notice that the principled std comes out to about 0.30, while our hand-tuned value in Chapter 11 was 0.20. Close, but not exact. With the principled value, the validation loss reproduces the same ~2.10 we got from hand-tuning, but now we have a formula that scales to any depth.

> **Why this is principled rather than guessed**
> The Kaiming formula is derived by assuming the inputs are Gaussian with unit variance, computing what the variance of the output of (linear + non-linearity) will be, and solving for the weight scale that keeps the output variance equal to one. The derivation is a few lines of algebra and is in the original 2015 paper. Critically, the formula scales: it tells you the correct std for *any* layer with *any* fan_in, not just our specific case. PyTorch provides `torch.nn.init.kaiming_normal_` as a one-line implementation.

The PyTorch convention

`torch.nn.init.kaiming_normal_(tensor, mode='fan_in', nonlinearity='tanh')` implements this formula directly. `nonlinearity` specifies which gain to use; PyTorch maintains a table mapping non-linearity names to their gains. For tanh the gain is 5/3; for ReLU it is sqrt(2); for linear it is 1; for sigmoid it is 1; for leaky ReLU it depends on the negative slope. The exact value of these gains is computed from the integral of the non-linearity's squared output over a unit Gaussian input — you do not need to memorize the derivation, just the existence of the table.

14. The tanh gain of 5/3 - where does it come from?

Chapter overview

The number 5/3 = 1.667 appears without explanation in the Kaiming formula for tanh. This chapter explains where it comes from, because once you understand the derivation, you can compute the gain for any new non-linearity yourself rather than looking it up.

Imagine you have a Gaussian input to `tanh` with mean 0 and standard deviation 1. The output of `tanh` is no longer Gaussian (it is bounded by –1 and 1), and its standard deviation is smaller than 1 because the tails got squashed. How much smaller? About 0.628. So tanh acts as a contractor that shrinks the standard deviation by roughly 0.628.

To preserve the standard deviation across (linear + tanh), the linear layer must *expand* the standard deviation by the inverse of 0.628, which is approximately 1.59 ≈ 5/3. That is where the gain comes from: it is precisely the amount by which the linear layer must expand the input to cancel out the contraction that tanh will apply.

> **Computing the gain numerically**
> If you ever invent a new non-linearity and want to find its Kaiming gain, the procedure is: sample many values from a standard Gaussian, apply your non-linearity, compute the standard deviation of the result, and take its reciprocal. That is your gain. The formula for tanh (5/3), for ReLU (sqrt(2)), and for the others in PyTorch's table are all computed this way; the closed-form expressions exist but the numerical computation is just as valid and works for arbitrary non-linearities.

> **Why this matters for the lecture**
> Karpathy emphasizes that the precise value of the gain matters less today than it once did, because modern innovations like Batch Normalization (introduced next) make the network much less sensitive to the exact initialization scale. But for very deep networks without BatchNorm — and for understanding why pre-2015 deep learning was so finicky — the gain is part of the story.

15. Why init alone will not scale to deep networks

Chapter overview

Kaiming initialization solves the problem of preserving std across one linear+nonlinearity block. For a two-layer MLP like ours, this is enough. But once we want to stack 10 or 50 or 100 layers, the precision required of initialization becomes unrealistic. This chapter explains why, which motivates the introduction of Batch Normalization in the next chapter.

The Kaiming formula is exact only if the assumptions hold: inputs are Gaussian, the non-linearity is exactly tanh (or exactly ReLU), the layers are simple linear+nonlinearity, and there is no interaction with other architectural elements. In a real network with residual connections, convolutional layers, attention, dropout, multiple non-linearities of different types, and so on, the assumptions break and the std slowly drifts away from 1 as the signal propagates.

In a 50-layer network, even a 5% drift per layer compounds to a factor of $1.05^{50} \approx 11$ — an order of magnitude. The activations either blow up or vanish, depending on the direction of the drift, and the network stops training. Adjusting the scaling factor by hand for each layer is not practical for serious research-scale models.

The fundamental observation behind BatchNorm

The Bengio insight that motivates Batch Normalization is simple and forceful: if we want the activations at every layer to have a particular distribution (specifically, zero mean and unit standard deviation), why not just *force* them to have that distribution? Take the activations, compute their mean and standard deviation across the batch, and normalize them. The operation is differentiable, so it can be inserted into the network and trained end-to-end. The network no longer depends on the initialization being exactly right; the normalization layer enforces the right statistics regardless.

16. The BatchNorm insight: just normalize the activations

Chapter overview

Batch Normalization, introduced in *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift* by Ioffe and Szegedy (2015), is one of the most impactful innovations in deep learning. It made it possible to train very deep networks routinely, removed the need for delicate per-layer initialization tuning, and turned out to have a regularization side-effect as well. This chapter introduces the core idea before we look at the implementation.

The idea is short and bold: take the pre-activations going into your non-linearity, compute their mean and standard deviation across the batch, subtract the mean, and divide by the standard deviation. The result is, by construction, a batch of values with mean 0 and standard deviation 1 — whether or not the upstream initialization happened to produce that distribution.

Karpathy on the first reading

“When I first read it my mind was blown, because you can just normalize these hidden states. Standardizing them so they are unit Gaussian is a perfectly differentiable operation. If you want unit Gaussian states in your network, just normalize them.”

Why it works

Before Batch Normalization, the only way to control the distribution of activations was to carefully choose the weight initialization, and any subsequent training step could push the distribution away from the desired range. BatchNorm sidesteps this entirely: no matter what the upstream weights do, the normalization layer brings the activations back to mean 0 and standard deviation 1 before passing them to the non-linearity. This decouples the optimization of upstream layers from the statistical behaviour of downstream layers.

Why the layer needs more than just normalization

Pure normalization is too restrictive. If we force every layer to output unit-Gaussian pre-activations, the network is constrained to a very specific shape and cannot, for instance, decide that a particular neuron should be saturated for some inputs (which is sometimes what we want). The original paper handles this with two learnable parameters per neuron: a **gain** (gamma) that scales the normalized value and a **bias** (beta) that shifts it. These give the network the freedom to undo the normalization if doing so produces a better loss. We will see them in the next chapter's code.

17. The forward pass: mean, std, gamma, beta

Chapter overview

This chapter shows the actual BatchNorm forward pass, inserted into our training loop between the linear layer and the tanh non-linearity. The mathematical operation is short; the surrounding bookkeeping (running statistics for inference) is the larger part of the code. We focus on the core math here and cover the bookkeeping in Chapters 18-19.

Cell - BatchNorm parameters and the forward pass

```
# BatchNorm parameters
bngain = torch.ones((1, n_hidden))
bnbias = torch.zeros((1, n_hidden))
bnmean_running = torch.zeros((1, n_hidden))
bnstd_running = torch.ones((1, n_hidden))

# ... inside the training loop, between Linear and Tanh ...
hpreact = embcat @ W1   # + b1 (we will drop b1 next chapter)

# BatchNorm layer
bnmeani = hpreact.mean(0, keepdim=True)
bnstdi  = hpreact.std(0,  keepdim=True)
hpreact = bngain * (hpreact - bnmeani) / bnstdi + bnbias

with torch.no_grad():
    bnmean_running = 0.999 * bnmean_running + 0.001 * bnmeani
    bnstd_running  = 0.999 * bnstd_running  + 0.001 * bnstdi

# Non-linearity
h = torch.tanh(hpreact)
```

Reading the forward pass line by line

- `bnmeani = hpreact.mean(0, keepdim=True)` computes the mean over the batch dimension (dim 0) for each of the 200 hidden units separately. Result shape: (1, 200).
- `bnstdi = hpreact.std(0, keepdim=True)` computes the standard deviation the same way. Result shape: (1, 200).
- `(hpreact - bnmeani) / bnstdi` is the actual normalization. Each example's pre-activation for each neuron is centred and scaled so that, across the batch, each neuron has mean 0 and std 1.

- `bngain * normalized + bnbias` applies the learnable gain and bias. Initialized to 1 and 0 respectively, so at the very first step BatchNorm is the identity on the normalized values. During training, the network can move these parameters wherever it wants.

> **Why the batch dimension and not the feature dimension**
> BatchNorm normalizes *across the batch*, not *across features*. For each hidden unit (each of the 200 columns), we compute the mean and std of that unit's pre-activations over the 32 examples in the batch, and normalize that column. Different hidden units have independent normalization. This is what makes BatchNorm couple examples together — the normalization of example A's activations depends on what example B's activations happened to be in the same batch. This coupling has consequences we will revisit in Chapter 20.

18. Running statistics for inference

Chapter overview

BatchNorm's reliance on batch statistics raises an immediate question: what happens at inference time, when we might want to predict for a single example with no batch? We cannot compute a meaningful batch mean and std from one sample. The solution is to maintain a running estimate of the mean and std during training, and use those running values at inference time. This chapter explains how the running statistics are computed and why.

Looking at the relevant lines from the forward pass:

```
with torch.no_grad():
    bnmean_running = 0.999 * bnmean_running + 0.001 * bnmeani
    bnstd_running  = 0.999 * bnstd_running  + 0.001 * bnstdi
```

On every training step, after computing the current batch's mean and std, we update the running estimate as a weighted average: 99.9% of the old running estimate plus 0.1% of the new batch estimate. This is called an **exponential moving average**, and the weighting factor (0.001 here) is called the **momentum**. After enough training steps, the running mean and std converge to good estimates of the population mean and std over the entire training set.

> **Why we use no_grad here**
> `with torch.no_grad():` tells PyTorch that the operations inside the block should not be tracked for gradient computation. The running statistics are not parameters that we want to update via gradient descent; they are buffers that track a running average over time. We do not want the computation graph to extend through these updates, and we do not want gradients to flow into them. Marking them no_grad keeps the graph clean and prevents accidental backpropagation through statistics that should be treated as constants.

Cell - An alternative: calibrate after training

```
# calibrate the batch norm at the end of training
with torch.no_grad():
    # pass the training set through
```

```
emb = C[Xtr]
embcat = emb.view(emb.shape[0], -1)
hpreact = embcat @ W1
# measure the mean/std over the entire training set
bnmean = hpreact.mean(0, keepdim=True)
bnstd  = hpreact.std(0,  keepdim=True)
```

Karpathy shows an alternative: after training is finished, pass the entire training set through the network once and compute the exact mean and std of the pre-activations. This is more accurate than the running average, but it requires a second pass over the data and adds complexity. The running-average approach is the standard choice in practice because it folds the calibration into training without an extra stage.

19. Train mode vs eval mode

Chapter overview

Because BatchNorm behaves differently during training (using batch statistics) versus inference (using running statistics), every BatchNorm layer needs to know what mode it is in. PyTorch and every other framework support this through a per-layer `training` flag that is set to `True` during training and `False` during inference. This chapter explains the mechanism and the consequences of getting it wrong.

In the manual implementation, the train/eval distinction is implicit:

```
# At training time, inside the loop:
hpreact = bngain * (hpreact - bnmeani) / bnstdi + bnbias
# - uses bnmeani, bnstdi from the current batch

# At inference time, outside the loop:
hpreact = bngain * (hpreact - bnmean_running) / bnstd_running + bnbias
# - uses bnmean_running, bnstd_running accumulated during training
```

The forward pass formula is the same; only the mean and std being subtracted change. At training time we want batch statistics so that the gradient flowing back through the normalization is correct. At inference time we want population statistics so that predictions are deterministic (the same input always gives the same output, regardless of what other inputs happen to be in the batch).

> **A classic source of bugs**
> Forgetting to switch a model between train mode and eval mode is one of the most common bugs in deep learning. The PyTorch convention is `model.eval()` before inference and `model.train()` when resuming training. If you forget, your validation loss can be wildly different from your training loss not because of overfitting, but because BatchNorm is silently using the wrong statistics. Always check that you have toggled the mode before suspecting more exotic explanations for unexpected losses.

20. Caveats and alternatives (LayerNorm, GroupNorm)

Chapter overview

BatchNorm was a breakthrough and it works well most of the time, but it has known drawbacks. Karpathy explicitly says “no one likes this layer” in the lecture, and a generation of researchers has developed alternatives that avoid its problems. This chapter summarizes the main drawbacks and lists the most important alternatives.

Drawback 1: example coupling

Inside a BatchNorm layer, the activations of any one example depend on the activations of all other examples in the same batch. This is a strange property for a forward pass to have, because it means the network's output for input A is different depending on what inputs B, C, D, etc. happened to be in the batch. In practice this turns out to act as a mild regularizer (a noise source that prevents overfitting), but it also causes subtle bugs when batch composition matters.

Drawback 2: small batches are unreliable

When the batch size is small (say, 8 or fewer), the per-batch mean and std are noisy estimates of the population statistics. The running estimates take longer to converge, and the activations themselves fluctuate more during training. This is why BatchNorm performs badly on very small batches and why distributed training with per-GPU batches that are small has to use either synchronized BatchNorm or a different normalization scheme entirely.

Drawback 3: variable-length sequences

BatchNorm assumes every element of the batch has the same shape. For sequence data of varying length (text, audio, video), naively applying BatchNorm requires padding and masking, which complicates the implementation and can produce subtly wrong normalization statistics.

Modern alternatives

- **LayerNorm** normalizes across the feature dimension within each example, not across the batch dimension. This eliminates example coupling and removes the train/eval distinction. LayerNorm is the dominant choice in Transformers and is what GPT, BERT, and the rest of the modern NLP stack use.
- **GroupNorm** splits the features into groups and normalizes within each group, independently per example. Used heavily in computer vision when batch sizes are small (object detection, segmentation).
- **InstanceNorm** normalizes each example independently across spatial dimensions, used in style transfer and certain image generation tasks.
- **RMSNorm** is a simplification of LayerNorm that omits the mean subtraction. Cheaper to compute and is used in LLaMA and several other modern LLMs.

Why BatchNorm is still worth learning

Despite the drawbacks and the existence of better alternatives, BatchNorm remains the conceptual starting point for all the normalization layers that came after. Understanding why it works and what its failure modes are gives you the framework to understand LayerNorm, GroupNorm, and any future normalization layer that gets invented. And it is still the right choice in many computer vision settings with large batches.

PART IV

Building deeper networks, PyTorch-style

Refactoring the code into reusable modules and stacking them into a six-layer MLP that looks like real PyTorch.

21. Wrapping operations as modules: Linear, BatchNorm1d, Tanh

Chapter overview

Up to this point our code has been imperative: every forward pass is written out inline as a sequence of operations on raw tensors. This is fine for one or two layers but rapidly becomes unmanageable as networks get deeper. The PyTorch convention is to wrap each kind of operation (linear, BatchNorm, activation) into a class with two methods: a constructor that creates the parameters, and a `__call__` method that performs the forward pass. Karpathy reimplements three such classes from scratch, and the resulting API is identical to `nn.Linear`, `nn.BatchNorm1d`, and `nn.Tanh` in PyTorch proper.

Cell - The Linear class

```
class Linear:
    def __init__(self, fan_in, fan_out, bias=True):
        self.weight = torch.randn((fan_in, fan_out), generator=g) / fan_in**0.5
        self.bias = torch.zeros(fan_out) if bias else None

    def __call__(self, x):
        self.out = x @ self.weight
        if self.bias is not None:
            self.out += self.bias
        return self.out

    def parameters(self):
        return [self.weight] + ([] if self.bias is None else [self.bias])
```

The `Linear` class is a thin wrapper around the linear operation we have been doing all along. The constructor takes `fan_in` and `fan_out`, initializes the weights using the Kaiming scaling (with gain 1 by default, gain 5/3 if used before tanh), and optionally creates a zero bias. The `__call__` method does the forward computation. The `parameters` method returns the trainable tensors for inclusion in the optimizer's parameter list.

Cell - The BatchNorm1d class

```
class BatchNorm1d:
    def __init__(self, dim, eps=1e-5, momentum=0.1):
        self.eps = eps
        self.momentum = momentum
        self.training = True
        # parameters (trained with backprop)
        self.gamma = torch.ones(dim)
        self.beta  = torch.zeros(dim)
        # buffers (trained with a running 'momentum update')
        self.running_mean = torch.zeros(dim)
        self.running_var  = torch.ones(dim)

    def __call__(self, x):
        if self.training:
            xmean = x.mean(0, keepdim=True)
            xvar  = x.var(0,  keepdim=True)
```

```
        else:
            xmean = self.running_mean
            xvar  = self.running_var
        xhat = (x - xmean) / torch.sqrt(xvar + self.eps)
        self.out = self.gamma * xhat + self.beta
        if self.training:
            with torch.no_grad():
                self.running_mean = (1 - self.momentum) * self.running_mean + self.momentum * xmean
                self.running_var  = (1 - self.momentum) * self.running_var  + self.momentum * xvar
        return self.out

    def parameters(self):
        return [self.gamma, self.beta]
```

How the BatchNorm1d class implements everything from Chapters 17-19

- `gamma` **and** `beta` are the learnable gain and bias, initialized to 1 and 0 so that BatchNorm starts as the identity on normalized inputs.
- `running_mean` **and** `running_var` are the running statistics for inference, kept as buffers and updated each forward pass under `no_grad`.
- `self.training` is the mode flag. When True, batch stats are used and running stats are updated. When False, running stats are used and nothing is updated.
- `self.eps = 1e-5` is a small constant added inside the square root to prevent division by zero when the variance happens to be exactly zero (rare but possible).
- **The momentum is 0.1 here (PyTorch default).** Note this is the opposite convention from the earlier manual implementation, which used 0.001. PyTorch's momentum is the weight on the new sample; the earlier implementation's was the weight on the old running estimate.

Cell - The Tanh class

```
class Tanh:
    def __call__(self, x):
        self.out = torch.tanh(x)
        return self.out
    def parameters(self):
        return []
```

`Tanh` has no parameters, no state, no train/eval distinction. It is the simplest possible module — included only for the sake of uniformity, so that we can iterate over a list of layers without special-casing the activation functions.

22. The 6-layer deep MLP

Chapter overview

With the three classes in place, we can now build a much deeper network with very little code. Karpathy stacks six Linear+BatchNorm+Tanh blocks, ending with a Linear+BatchNorm to produce the logits. This is the deepest network we have built in the series, and the entire architecture fits into a single Python list.

Cell - The 6-layer network

```
n_embd = 10
n_hidden = 100
g = torch.Generator().manual_seed(2147483647)

C = torch.randn((vocab_size, n_embd), generator=g)
layers = [
  Linear(n_embd * block_size, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
  Linear(           n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
  Linear(           n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
  Linear(           n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
  Linear(           n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
  Linear(           n_hidden, vocab_size, bias=False), BatchNorm1d(vocab_size),
]
```

Five Linear-BatchNorm-Tanh blocks followed by a final Linear-BatchNorm. Each Linear has `bias=False` — we will explain why in the next chapter. The final block does not have a Tanh because we want raw logits as the output, not bounded values.

Initialization adjustments

Cell - Per-layer gain adjustments

```
with torch.no_grad():
    # last layer: make less confident
    layers[-1].gamma *= 0.1
    # all other layers: apply gain
    for layer in layers[:-1]:
        if isinstance(layer, Linear):
            layer.weight *= 1.0   # 5/3 if no batchnorm

parameters = [C] + [p for layer in layers for p in layer.parameters()]
print(sum(p.nelement() for p in parameters))
for p in parameters:
    p.requires_grad = True
```

Two initialization tweaks happen here. First, the gamma of the final BatchNorm is scaled by 0.1 — the equivalent of our Chapter 7 output-layer fix, but performed on the BatchNorm's gain parameter rather than on the Linear's weights. Second, each Linear layer's weights would be scaled by the tanh gain of 5/3 if we were not using BatchNorm; because BatchNorm is normalizing the pre-activations anyway, the gain matters less and we leave it at 1.0 here.

How the layers iterate during training

The forward pass through this stack is dramatically simpler than the inline version: `x = embeddings; for layer in layers: x = layer(x); loss = F.cross_entropy(x, Y)`. The whole network is one for-loop. Adding a new layer means appending to the list. Re-running with a different depth means changing the list construction. This is the same pattern that `nn.Sequential` implements in PyTorch.

23. Why bias=False before BatchNorm

Chapter overview

Look closely at every `Linear` call in the deep network: all of them have `bias=False`. This is not a typo. When a Linear layer is immediately followed by a BatchNorm layer, the Linear's bias is redundant — BatchNorm will subtract any constant offset out as part of its mean subtraction. Including the bias would not hurt correctness, but it would waste parameters and add unnecessary computation. This chapter explains the math.

Consider a Linear layer producing pre-activations `z = x @ W + b`, followed immediately by BatchNorm. BatchNorm subtracts the mean of `z` across the batch:

```
# Linear:    z = x @ W + b
# BatchNorm: znorm = (z - z.mean(0)) / z.std(0)

# But z.mean(0) = (x @ W).mean(0) + b
# Substituting:
#  znorm = (x @ W + b - (x @ W).mean(0) - b) / z.std(0)
#        = (x @ W       - (x @ W).mean(0)    ) / z.std(0)
#
# The two b's cancel. The bias has zero effect on znorm.
```

Because the bias is exactly cancelled by the mean subtraction, it cannot influence the output of the BatchNorm layer, which means it cannot influence the loss, which means its gradient is always exactly zero, which means it never updates. It is a parameter that sits unused for the entire training run.

BatchNorm's beta replaces the bias

If the Linear's bias is doing nothing, you might worry that we have lost the ability to shift the output. We have not: BatchNorm's own `beta` parameter (initialized to 0, shape `(n_hidden,)`) provides exactly the same capability and is in the right place to actually have an effect. Karpathy treats this as a clean substitution: drop the Linear's bias, keep the BatchNorm's beta, get the same expressive power with fewer parameters.

A subtle but important convention

Whenever you place a Linear (or Conv2d) immediately before a BatchNorm, set `bias=False` on the linear layer. It is one of those small details that marks the difference between code written by someone who knows what they are doing and code written by someone who copy-pasted a template. PyTorch's `nn.Linear` supports `bias=False` as a constructor argument for exactly this reason.

PART V

Diagnostic tools and conclusion

The plots that tell you whether your neural network is training in a healthy state, plus summary and reference.

24. Forward activation histograms

Chapter overview

With a six-layer network in hand, we now learn how to inspect what is happening inside it. The first diagnostic is the histogram of activations at each layer. A healthy network has roughly similar activation distributions at every depth; an unhealthy one shows distributions that grow, shrink, or saturate as the signal propagates. This chapter explains the diagnostic and what to look for.

Cell - Plot activation histograms per layer

```
plt.figure(figsize=(20, 4))
legends = []
for i, layer in enumerate(layers[:-1]):
    if isinstance(layer, Tanh):
        t = layer.out
        print('layer %d (%10s): mean %+.2f, std %.2f, saturated: %.2f%%' %
              (i, layer.__class__.__name__, t.mean(), t.std(),
               (t.abs() > 0.97).float().mean() * 100))
        hy, hx = torch.histogram(t, density=True)
        plt.plot(hx[:-1].detach(), hy.detach())
        legends.append(f'layer {i} ({layer.__class__.__name__}')
plt.legend(legends);
plt.title('activation distribution')
```

What a healthy plot looks like

After every Tanh layer in the network, we plot a histogram of the output values. In a well-tuned network, all five histograms overlap almost perfectly — the activation distribution is essentially the same shape and width at every depth. Mean stays near zero, std stays near 0.65 (the natural std of tanh applied to a unit Gaussian), and the saturation percentage stays under about 20%.

What a broken plot looks like

If initialization is wrong — for example, gain set too high — the activation distributions shift across layers. Each successive layer has a wider or narrower distribution than the previous. After a few layers, activations are either saturated (most pixels at ±1) or vanishing (most pixels near zero). Gradients flowing back through this network will be tiny or huge depending on which direction the drift went, and training will stall.

> **What 'saturated' means in the printed output**
> The print statement reports the fraction of activations with |value| > 0.97 — the threshold for "effectively at the tanh asymptote." Anything above about 20% means a non-trivial number of neurons are saturated and not contributing to learning. Above 50% means the network is in serious trouble. A well-initialized network with BatchNorm typically sits around 5-15%.

25. Backward gradient histograms

Chapter overview

The activation histograms tell us how the forward pass is behaving. The mirror image — the gradient histograms — tells us how the backward pass is behaving. We want gradients to have roughly the same magnitude at every depth. Symmetry across layers indicates that the chain-rule products are not amplifying or attenuating too much. This chapter introduces the diagnostic.

Cell - Plot gradient histograms per layer

```
plt.figure(figsize=(20, 4))
legends = []
for i, layer in enumerate(layers[:-1]):
    if isinstance(layer, Tanh):
        t = layer.out.grad
        print('layer %d (%10s): mean %+f, std %e' %
              (i, layer.__class__.__name__, t.mean(), t.std()))
        hy, hx = torch.histogram(t, density=True)
        plt.plot(hx[:-1].detach(), hy.detach())
        legends.append(f'layer {i} ({layer.__class__.__name__}')
plt.legend(legends);
plt.title('gradient distribution')
```

What healthy and unhealthy gradients look like

In a well-initialized network, the gradient histograms at every depth overlap, just like the activation histograms. Each Tanh layer's `.out.grad` has roughly the same standard deviation. This means the backward signal is reaching every layer with comparable magnitude, and every layer has a chance to learn.

In a badly initialized network, the gradient std at one depth differs from the gradient std at another by orders of magnitude. The classic failure mode is **vanishing gradients**: the layers closest to the input have gradients near zero, so they barely update. The other classic failure is **exploding gradients**: the early layers have gradients so large that updates push them to extreme values within a few steps. Both kill training.

> **Why this diagnostic is uniquely useful**
> The activation histograms reveal forward-pass problems but cannot directly diagnose vanishing or exploding gradients. The gradient histograms reveal the exact thing that determines whether the optimizer is actually able to make progress on a given parameter. Together, the two histograms give a complete picture of whether the forward and backward passes are both functioning correctly.

26. Weight gradient distribution and the last layer problem

Chapter overview

Beyond the per-layer gradient flow, it is useful to look at the gradients of the weight matrices themselves. This diagnostic reveals two common pathologies: weight gradients that have unusual shapes (e.g. heavy tails), and the “last layer problem” where the output layer's gradients are much larger than every other layer's. Karpathy demonstrates both.

Cell - Plot weight gradient distributions

```
plt.figure(figsize=(20, 4))
legends = []
for i, p in enumerate(parameters):
    t = p.grad
    if p.ndim == 2:
        print('weight %10s | mean %+f | std %e | grad:data ratio %e' %
              (tuple(p.shape), t.mean(), t.std(), t.std() / p.std()))
        hy, hx = torch.histogram(t, density=True)
        plt.plot(hx[:-1].detach(), hy.detach())
        legends.append(f'{i} {tuple(p.shape)}')
plt.legend(legends)
plt.title('weights gradient distribution');
```

Reading the grad-to-data ratio

The printed output includes `t.std() / p.std()`, the ratio of the gradient standard deviation to the data standard deviation. This is a rough measure of how aggressively SGD will modify each weight matrix. Most layers in a healthy network have a ratio around 1e-3, meaning the gradient is about a thousand times smaller than the data — sensible for a learning rate of 0.1 and a desired per-step relative change of about 1e-4.

The last layer problem

Karpathy points out that the output layer typically has a much larger grad-to-data ratio than the other layers — around 1e-2 instead of 1e-3, ten times larger. The reason is that the output layer interacts directly with the loss, so its gradient receives the full unattenuated error signal. If we used the same learning rate for all layers, the output layer would update ten times faster than the rest. In practice this either self-corrects after a few hundred steps, or you address it with a modern optimizer like Adam that uses per-parameter learning rates.

> **What Karpathy says**
> “The standard deviations are roughly 1e-3 throughout except for the last layer, which has roughly 1e-2 standard deviation of gradients. So the gradients on the last layer are currently about 10 times greater than all the other weights inside the neural net. That is problematic because in the simple SGD setup, you would be training this last layer about 10 times faster than you would be training the other layers at initialization.”

27. The update-to-data ratio - the most important plot

Chapter overview

Of all the diagnostics Karpathy introduces, the update-to-data ratio is the one he calls out as the most important. It tells you exactly how much each weight changes per training step relative to its current magnitude. The healthy value is around **1e-3** on a log scale, meaning each step changes the weights by about 0.1% of their current size. This chapter shows the diagnostic, explains its interpretation, and explains why it is more informative than the grad-to-data ratio from the previous chapter.

Cell - Track the update-to-data ratio over time

```
# inside the training loop, after the update step:
with torch.no_grad():
    ud.append([((lr * p.grad).std() / p.data.std()).log10().item()
               for p in parameters])
```

For each parameter, we compute `lr * p.grad` — the actual update that will be applied to `p.data`. We then take the std of that update and divide by the std of the data, and log it. We track this list across the entire training run and then plot it.

Cell - Plot the update-to-data ratio over time

```
plt.figure(figsize=(20, 4))
legends = []
for i, p in enumerate(parameters):
    if p.ndim == 2:
        plt.plot([ud[j][i] for j in range(len(ud))])
        legends.append('param %d' % i)
plt.plot([0, len(ud)], [-3, -3], 'k')   # horizontal line at the target value
plt.legend(legends);
```

The plot has one line per weight matrix, showing how the log10 of the update-to-data ratio evolves over training. A horizontal reference line at –3 marks the target. Lines hovering near –3 indicate healthy training. Lines significantly above (say, –2) mean updates are too aggressive; lines significantly below (say, –4) mean the learning rate is too small for those parameters.

> **Why update-to-data is more informative than grad-to-data**
>
> The grad-to-data ratio tells you the relative magnitude of the gradient. But what actually changes the data is `lr * grad`, not `grad`. If different layers want different effective step sizes, a single learning rate that is right for one layer will be wrong for another. The update-to-data ratio bakes the learning rate into the measurement and reveals the actual change being applied. It is the diagnostic that tells you whether your learning rate is well-tuned for each individual parameter.

> **The 1e-3 rule of thumb**
>
> “You want this ratio to be roughly 1e-3 on a log scale (so around -3 in the plot). If it is much higher than that, your learning rate is too high or the updates are too aggressive. If it is much lower, your learning rate is too small or that parameter is not learning. This is the single most useful diagnostic for tuning a learning rate.” This is also the reason modern optimizers like Adam, which use per-parameter learning rates, tend to work better than plain SGD — they automatically maintain something close to the right update-to-data ratio for every parameter.

28. Diagnostic cards: recognising broken training

Chapter overview

The diagnostic plots of the previous four chapters are powerful but require interpretation. This chapter consolidates the most common training pathologies into stand-alone diagnostic cards, each of which describes a symptom (what you see in the plots), the diagnosis (what is going wrong internally), and the action (how to fix it). Keep these handy — you will encounter every one of them at some point in your career.

Pathology 1 · Confidently wrong at init	
Symptom	Initial loss is much higher than log(K) for a K-way classifier. A “hockey-stick” loss curve in the first few hundred iterations.
Diagnosis	Output layer weights are too large at initialization, producing logits with wide spread, which softmax turns into sharp but random probability distributions.
Action	Scale the output Linear's weights down by a factor like 0.01, and initialize its bias to zero. The initial loss should now be close to log(K).

Pathology 2 · Saturated activations	
Symptom	Activation histogram is U-shaped, with most values piled up at ±1 (tanh) or 0 and 1 (sigmoid). High saturation percentage in printed stats.
Diagnosis	Hidden pre-activations have too large a standard deviation, pushing the non-linearity into its flat region where the gradient is near zero.
Action	Apply Kaiming initialization to the upstream weights with the correct gain for the non-linearity, or insert a BatchNorm layer before the non-linearity.

Pathology 3 · Dead neurons	
Symptom	Saturation map (an image of |activation| > threshold) has fully white columns. Particular neurons never produce gradient signal.
Diagnosis	For tanh / sigmoid: pre-activation is in the saturated tail for every example. For ReLU: pre-activation is negative for every example. The neuron cannot learn.
Action	Improve initialization. For ReLU, lower the learning rate to avoid knocking neurons into the dead region. Consider Leaky ReLU or ELU instead of ReLU.

Pathology 4 · Vanishing or exploding gradients	
Symptom	Backward gradient histograms have wildly different magnitudes at different depths. Early layers (closest to input) train glacially or not at all.
Diagnosis	Standard deviation drifts across layers because initialization does not preserve it, or because non-linearities are saturated.
Action	Fix initialization with the correct gain, add BatchNorm layers, or in very deep networks add residual connections.

Pathology 5 · Last layer dominating updates	
Symptom	Grad-to-data and update-to-data ratios for the output layer are 10× or more larger than for the hidden layers.
Diagnosis	The output layer interacts directly with the loss and receives the unattenuated gradient. Plain SGD with one learning rate cannot serve both regimes.
Action	Use a modern optimizer like Adam that maintains per-parameter step sizes, or apply a smaller multiplier to the output layer's learning rate.

Pathology 6 · Update-to-data ratio off	
Symptom	Update-to-data plot lines hover far from the –3 reference. Above –2 = updates too big; below –4 = updates too small.
Diagnosis	Learning rate is mis-tuned for the current parameter scales. Either weights are smaller/larger than the optimizer is expecting, or the LR was chosen without checking.
Action	If too high, multiply the learning rate down until the ratio returns toward 1e-3. If too low, multiply up. Repeat the LR finder from Part 2 Chapter 20 if necessary.

29. Summary, the loss log, and what Part 4 will add

Chapter overview

Time to consolidate. This chapter records the cumulative improvement we got from each fix, lists the most important takeaways from the entire lecture, and previews what the next lecture in the series introduces.

The loss log

Karpathy keeps a running “loss log” at the bottom of the notebook, recording the validation loss after each major change. The progression tells the story of the lecture:

Configuration	Train loss	Val loss
Original (Part 2 baseline, magic-number init)	2.124	2.168
Fix 1: squash output layer (W2 * 0.01, b2 = 0)	2.07	2.13
Fix 2: squash hidden pre-activations (W1 * 0.2)	2.036	2.103
Principled Kaiming init (5/3 / sqrt(fan_in))	2.038	2.107
Add Batch Normalization layer	**2.067**	**2.105**

The big takeaway is that the actual *numerical* improvement from BatchNorm here is small — from 2.107 to 2.105 — because our network is shallow enough that good initialization alone gets us most of the way. The point of BatchNorm in this lecture is not to win a tenth of a loss point. It is to introduce the technique and to demonstrate the diagnostic toolkit that lets us understand *why* networks behave the way they do.

The seven things to take away

- **1. Sanity-check your initial loss.** It should be close to log(K) for a K-way classifier. If it is much higher, you are confidently wrong at init — fix the output layer first.
- **2. Plot activation histograms.** They should be roughly the same shape at every depth. U-shaped (saturated) or collapsed (vanished) means broken initialization.
- **3. Plot gradient histograms.** They should also be roughly the same shape at every depth. Drift signals vanishing or exploding gradients.
- **4. Track the update-to-data ratio.** It should hover near 1e-3 on a log scale. Above or below indicates a mis-tuned learning rate for that parameter.
- **5. Kaiming initialization is the principled choice.** `std = gain / sqrt(fan_in)`, with gain depending on the non-linearity. PyTorch implements this for you in `torch.nn.init`.
- **6. BatchNorm normalizes activations to unit Gaussian.** Learnable gain and bias make it as expressive as needed. Running mean and std are tracked for inference. Toggle train/eval mode appropriately.
- **7. When Linear is followed by BatchNorm, set bias=False.** The Linear's bias is cancelled out by BatchNorm's mean subtraction. It is dead weight.

Where Part 4 picks up

Part 4 of the makemore series (*Building makemore Part 4: Becoming a Backprop Ninja*) takes everything we have built and asks the next logical question: what if you had to implement the backward pass by

hand, with no PyTorch autograd? The exercise forces you to derive the analytical gradient of every operation in our network — including BatchNorm, which has a famously intricate backward pass. The reward is a much deeper intuition about why certain initialization and normalization choices help, because you will have seen the gradient formulas first-hand. The lecture also walks through the rest of the way to a small Transformer, taking what we have built and stacking architectural innovations on top of it.

> **The single thing to remember if you remember nothing else**
> Modern deep learning is a craft of controlling activation and gradient statistics. Every innovation that made deep networks routinely trainable — Kaiming init, BatchNorm, residual connections, layer norm, modern optimizers — addresses the same underlying problem: keeping the forward and backward statistics well-behaved as the network gets deeper. Once you can read activation and gradient histograms, you can understand *why* each of these innovations exists, and you can debug your own networks when they go wrong.

30. Glossary

This glossary defines every technical term used in the handbook in a single, scannable place. Look up anything you do not recognize here first.

Activation. The output of a non-linearity such as tanh, sigmoid, or ReLU. Distinguished from *pre-activation*, which is the value going *into* the non-linearity.

Activation histogram. A diagnostic plot showing the distribution of activation values at a given layer. A healthy network has similar histograms at every layer; an unhealthy one shows drift.

Batch Normalization (BatchNorm). A layer introduced by Ioffe and Szegedy in 2015 that normalizes pre-activations across the batch to unit Gaussian, then applies a learnable gain and bias. Made deep networks routinely trainable.

Bias (in a linear layer). The additive constant in `y = x @ W + b`. Set to `bias=False` when followed by BatchNorm, because BatchNorm's mean subtraction cancels it out.

Confidently wrong. Initial state of a neural network where random initialization produces sharp probability distributions, leading to a much higher initial loss than the uniform-prediction baseline of log(K).

Dead neuron. A neuron whose pre-activation is in the flat region of its non-linearity for every input in the dataset, so it produces no gradient signal and cannot learn. Permanent for ReLU; nearly so for tanh and sigmoid.

Eps (epsilon). A tiny constant added inside the square root of the variance in BatchNorm to prevent division by zero. PyTorch default: 1e-5.

Eval mode. The mode in which a model uses running statistics in its BatchNorm layers rather than batch statistics. Set with `model.eval()` in PyTorch before doing inference.

Fan-in / fan-out. The number of inputs (fan-in) and outputs (fan-out) to a linear layer. Kaiming initialization uses fan-in to scale the weight standard deviation.

Gain (in initialization). The non-linearity-specific multiplicative constant in the Kaiming formula. tanh: 5/3. ReLU: sqrt(2). Linear/sigmoid: 1. Determines how much the linear layer must expand standard deviation to cancel the non-linearity's contraction.

Gamma and beta (BatchNorm). The learnable gain and bias of a BatchNorm layer. Initialized to 1 and 0 respectively so the layer starts as the identity on normalized inputs. Optimizable with backprop like any other parameter.

GroupNorm. An alternative to BatchNorm that normalizes across groups of features within each example, not across the batch. Used in computer vision with small batches.

Hockey-stick loss curve. A loss curve where the first few hundred iterations show a steep drop from a very high initial loss to a normal value. Signature of confidently-wrong initialization.

hpreact. Karpathy's name for the pre-activation of the hidden layer — the value going into tanh. Isolated as its own variable so we can inspect its statistics.

Internal covariate shift. The original motivation given in the BatchNorm paper for why deep networks are hard to train: the distribution of inputs to each layer keeps changing as upstream weights update. The term is now considered somewhat misleading; BatchNorm's actual benefits come from smoother loss landscapes.

Kaiming (He) initialization. Drawing weights from a distribution with standard deviation `gain / sqrt(fan_in)`, where gain depends on the following non-linearity. Introduced in He et al. 2015.

LayerNorm. An alternative to BatchNorm that normalizes across the feature dimension within each example, not across the batch. Dominant in Transformers.

Logits. The raw, pre-softmax output of a classifier. Real-valued, can be positive or negative.

Momentum (BatchNorm). The weight on the new batch statistics in the exponential moving average that updates the running mean and variance. PyTorch default: 0.1. The lecture's manual implementation uses 0.001, but with the conventions reversed.

nn.Module. PyTorch's base class for layers and models. Provides parameter tracking, mode toggling (train/eval), and the `parameters()` method. The classes we build in Chapter 21 mirror this API.

Non-linearity (activation function). A function applied element-wise between linear layers, such as tanh, sigmoid, or ReLU. Without one, stacking linear layers is equivalent to a single linear layer.

Pre-activation. The value going into a non-linearity, before it has been applied. Distinguished from *activation*, which is the output of the non-linearity. The quantity BatchNorm normalizes.

PyTorchify. Karpathy's verb for refactoring imperative tensor code into reusable module classes that mirror PyTorch's `nn.Module` API. The work of Chapter 21.

ReLU. The non-linearity `max(0, x)`. Most popular activation function in modern deep learning, but susceptible to permanent dead neurons when the pre-activation is always negative.

Running mean / running variance. Exponential moving averages of the batch statistics, maintained by BatchNorm during training and used in place of batch statistics during inference.

Saturated. Describes a neuron whose activation is in the flat region of its non-linearity (e.g. |tanh| > 0.97). Gradients flowing through saturated neurons are near zero.

Saturation map. A diagnostic image showing which (sample, neuron) pairs are saturated. Fully white columns indicate dead neurons.

Train mode. The mode in which a model uses batch statistics in its BatchNorm layers and updates running statistics. Set with `model.train()` in PyTorch.

Update-to-data ratio. The standard deviation of `lr * grad` divided by the standard deviation of `data`, for a given parameter. Healthy value: approximately 1e-3 on a log scale. The most informative diagnostic for learning-rate tuning.

Vanishing / exploding gradients. The failure mode where gradients become tiny (vanishing) or huge (exploding) as they propagate backward through many layers. Diagnosed by per-layer gradient histograms. Fixed by careful initialization, BatchNorm, or residual connections.

End of Part 3. Part 4 walks through implementing the backward pass of every operation in this network by hand — including the famously intricate BatchNorm backward — so that you develop a true gradient-level intuition for why these architectural choices help. Same network, no autograd, full transparency.